

基于集成学习的恶意代码动态检测方法

刘强^{1,2}, 王坚¹, 王亚男¹, 王珊³

(1. 空军工程大学防空反导学院, 西安 710051; 2. 空军工程大学研究生院, 西安 710051; 3. 中国人民解放军 94789 部队, 南京 210018)

摘要:在当前网络环境中, 不断升级的恶意代码变种为网络安全带来了巨大挑战。现有的人工智能模型虽然在恶意代码检测方面成效明显, 但仍存在两个不可忽视的缺点。一是泛化能力较差, 虽然在训练数据上表现优异, 但受概念漂移现象的影响, 在实际测试中性能不够理想; 二是鲁棒性不佳, 容易受到对抗样本的攻击。为解决上述问题, 文章提出一种基于集成学习的恶意代码动态检测方法, 根据 API 序列的不同特征, 分别构建统计特征分析模块、语义特征分析模块和结构特征分析模块, 各模块针对性地进行恶意代码检测, 最后融合各模块分析结果, 得出最终检测结论。在 Speakeasy 数据集上的实验结果表明, 与现有研究方法相比, 该方法各项性能指标具有明显优势, 同时具有较好的鲁棒性, 能够有效抵抗针对 API 序列的两种对抗攻击。

关键词: 恶意代码检测; n-gram 算法; Transformer 编码器; 图神经网络; 对抗性攻击

中图分类号: TP309 文献标志码: A 文章编号: 1671-1122 (2025) 01-0159-14

中文引用格式: 刘强, 王坚, 王亚男, 等. 基于集成学习的恶意代码动态检测方法 [J]. 信息安全, 2025, 25 (1): 159-172.

英文引用格式: LIU Qiang, WANG Jian, WANG Yanan, et al. A Dynamic Malware Detection Method Based on Ensemble Learning[J]. Netinfo Security, 2025, 25(1): 159-172.

A Dynamic Malware Detection Method Based on Ensemble Learning

LIU Qiang^{1,2}, WANG Jian¹, WANG Yanan¹, WANG Shan³

(1. School of Air Defense and Antimissile, Air Force Engineering University, Xi'an 710051, China; 2. Graduate School of Air Force Engineering University, Xi'an 710051, China; 3. 94789 Troop of PLA, Nanjing 210018, China)

Abstract: In the current network environment, constantly upgrading variants of malicious code pose significant challenges to network security. Although existing artificial intelligence models have shown significant effectiveness in detecting malicious code, there are still two undeniable shortcomings. Firstly, their generalization ability is poor. Although they perform well on training data, their performance is not ideal in actual testing due to the phenomenon of concept drift. Secondly, their robustness is poor and they are susceptible

收稿日期: 2024-09-25

基金项目: 国家自然科学基金 [61806219, 61703426, 61876189]; 陕西省高校科协青年人才托举计划 [20190108, 20220106]; 陕西省创新能力支撑计划 [2020KJXX-065]

作者简介: 刘强 (1993—), 男, 陕西, 助理工程师, 硕士研究生, 主要研究方向为网络空间安全和恶意代码检测; 王坚 (1982—), 男, 陕西, 副教授, 硕士, 主要研究方向为智能信息处理和网络安全防护; 王亚男 (1988—), 女, 陕西, 讲师, 博士, 主要研究方向为网络信息安全和人工智能; 王珊 (1989—), 女, 江苏, 工程师, 硕士, 主要研究方向为信息通信技术。

通信作者: 王亚男 wyn1988814@163.com

to attacks from adversarial samples. To solve the above problems, this paper proposed a dynamic detection method for malicious code based on ensemble learning. According to the different features of API sequences, statistical feature analysis module, semantic feature analysis module, and structural feature analysis module were respectively constructed. Each module performed targeted malicious code detection, and finally integrated the analysis results of each module to obtain the final detection conclusion. The experimental results on the Speakeasy dataset show that compared with existing research methods, this method has significant advantages in various performance indicators and good robustness, which can effectively resist two adversarial attack methods against API sequences.

Key words: malware detection; n-gram algorithm; Transformer encoder; graph neural network; adversarial attack

0 引言

恶意代码通常指的是故意设计用来破坏计算机系统功能、搜集敏感信息、窃取用户隐私或对网络服务造成干扰的代码或其载体。它们可能以病毒、蠕虫、木马、间谍软件或勒索软件等多种形式存在,每种恶意代码都有其独特的传播机制、感染方式和破坏模式。卡巴斯基安全网发布的《2023年移动恶意软件威胁形势》^[1]指出,其安全产品在2023年阻止了近3380万次恶意软件、广告软件和风险软件攻击,共检测到130多万个恶意安装包,其中15.4万个包含手机银行特洛伊木马。由此可见,在现代网络环境中,恶意代码的不断变异对用户的日常生活产生了严重影响,甚至危及国家的网络安全^[2]。随着5G和人工智能等先进技术的飞速发展,网络安全的重要性越发凸显^[3],对未知恶意代码及其变种进行快速准确地识别,是有效防范恶意攻击的基础和前提^[4]。因此,高效地开展恶意代码检测工作显得尤为重要,这不仅有助于及时发现并防范潜在攻击,还能够为构建更加安全的网络空间提供坚实的基础。

恶意代码分析技术可分为静态分析和动态分析两种。静态分析技术不运行代码,直接对可疑的恶意文件进行分析,从中发现其特征、功能和潜在危害,因此检测速度快,但由于某些恶意软件使用加密和混淆等先进技术,静态分析并不能取得良好效果。动态分析技术在沙箱里执行可疑的恶意文件,期间所有行为都会受到监控和记录,因此不受加密和混淆的影响。

然而动态分析需要占用大量的计算和存储资源,分析的难度和成本较高,此外,恶意软件不断开发新的反分析技术来逃避分析工具,如果恶意软件检测到正在被分析,就会隐藏其恶意意图^[5],因此动态分析也存在误报和漏报的可能性。

近年来,随着人工智能技术的发展,机器学习和深度学习各项技术已被广泛应用到恶意代码检测领域,这些技术通过智能提取恶意软件静态或动态特征,在检测已知和未知恶意软件方面都取得了一定成果。然而机器学习和深度学习技术也有其局限性,一方面,网络空间环境持续变化,新的0-day、N-day攻击等不断出现,导致测试数据分布随时间动态变化。由于测试数据的分布与原始训练数据的分布不一致,训练好的模型会产生严重的偏差,这种现象被称为概念漂移^[6];另一方面,人工智能模型本质上很容易受到对抗样本的攻击,攻击者只需通过修改恶意代码的少量字节,便可绕过模型的检测。

在现实网络环境中,恶意代码检测仍面临着诸多挑战,在各种攻击手段不断发展演变的背景下,如何提高恶意代码检测的性能,是一个需要持续研究的课题。为提高人工智能模型的鲁棒性,有效应对不断升级的恶意代码变种,本文提出一种基于集成学习的恶意代码动态检测方法,该方法根据API序列的统计特征、语义特征和结构特征,构建了3个模型分别进行针对性检测,融合检测结果后得出最终结果。本文主要创新点如下。

1) 提出了一种有效的集成学习方法,实现了高精

度的恶意代码检测。针对API序列的统计特征,构建了基于n-gram特征的随机森林分类器进行恶意代码检测;针对API序列的语义特征,构建了基于Transformer编码器的深度学习模型,实现恶意代码检测;针对API序列的结构特征,为每个序列生成有向图,采用图同构网络(Graph Isomorphism Network, GIN)模型,实现恶意代码检测。最后融合3个模型的输出概率,得出最终检测结果。

2) 采用两种现有方法进行对抗性攻击实验,验证了模型的鲁棒性。

1 相关工作

在动态分析中,研究人员提取和分析的动态特征包括系统应用程序编程接口(Application Programming Interface, API)、操作码和控制流等。由于软件执行过程中会调用许多系统API,这些API表征了软件的所有行为,包括网络访问、文件创建和修改等,因此API调用序列已成为恶意软件动态检测广泛使用的特征。

有的学者根据API序列的统计特征进行恶意软件检测,通过API调用的频率差异区分恶意软件和良性软件。QIAO^[7]等人将研究重点放在API调用序列中频繁消息的使用上,通过频繁项集挖掘和相似性计算来处理API名称和参数。在大型恶意软件数据集上的实验结果表明,仅使用API调用序列的频繁消息就可以实现高精度的恶意软件聚类,同时显著减少计算时间。UPPAL^[8]等人从API调用序列中提取4-gram特征,并通过频率统计来选择重要特征,将支持向量机(Support Vector Machine, SVM)作为分类模型。ALAZAB^[9]等人结合权限请求和API调用,提出了3种不同的分组策略来选择最有价值的API调用,以最大限度地提高识别Android恶意软件应用程序的可能性,3种分组分别为模糊组、风险组和破坏组。

有的学者根据API序列的语义特征进行恶意软件检测,通过API序列内部的依赖关系区分恶意软件和良性软件。KOLOSNAJI^[10]等人构建了一个基于卷积和递归网络层的神经网络,以获得最佳的分类特征。

他们通过这种方式,得到一个分层特征提取架构,将n元图的卷积与全序列建模相结合,在CNN之后,应用长短期记忆(Long Short-Term Memory, LSTM)网络来处理时间序列。AGRAWAL^[11]等人将API名称和字符串参数的n-gram特征输入几个堆叠的LSTM中。这种方法实现了比文献[10]所提方法更好的性能。ZHANG^[12]等人提出一种低成本的特征提取方法以及一种有效的深度神经网络架构,用于准确快速地检测恶意软件。特征表示方法利用特征哈希技巧对与API名称相关联的API调用参数进行编码,深度神经网络架构应用多个Gated-CNN来变换每个API调用的提取特征,随后通过双向LSTM对输出进行进一步处理,以学习API调用之间的顺序相关性。LI^[13]等人提出一种恶意软件检测框架,该框架使用深度学习模型来捕获和组合更有意义的API序列特征,首先使用嵌入层和卷积层对多个API进行联合表示,以表示软件行为;其次使用API的类别、动作和操作对象来表示每个API调用的语义信息;最后使用Bi-LSTM模块来挖掘API之间的关系信息。ZHANG^[14]等人融合API调用的语义和序列特征,提出了一种恶意软件检测模型Mal-ASSF。Mal-ASSF首先使用API2Vec嵌入方法获得API函数的降维表示,并使用Balts捕捉连续片段的序列特征,随后提取API函数的操作类型和资源类型,以挖掘API函数的隐含语义信息,最后将语义和序列特征通过注意力模块进行融合和处理。实验结果表明,Mal-ASSF在关键序列的语义表示和识别方面具有优越的能力。由于Transformer在处理长序列特征和理解上下文关系方面具有灵活性,因此近年来已有不少学者将Transformer模型应用于恶意代码检测领域。DEMIRKIRAN^[15]等人在恶意软件检测领域使用了预先训练的CANINE Transformer模型,并提出一种基于集成学习的模型RTF(随机Transformer森林),在4个API调用数据集上均取得了出色的分类效果。LI^[16]等人为了解决自注意力机制的计算效率问题,通过引入一种附加注意力机制来改进标准Transformer架构,提

高了恶意软件检测模型的训练效率。TRIZNA^[17]等人提出一种基于Transformer自注意力机制的神经网络架构Nebula,从动态日志报告中提取API调用、内存访问记录和文件操作等特征用于恶意软件动态分析,在3个不同数据集上进行实验,取得了比CNN和LSTM更好的检测和分类效果。

也有学者根据API序列的结构特征进行恶意软件检测,将API序列转换为有向图结构,通过分析图的整体特征来区分恶意软件和良性软件。JIANG^[18]等人提出一种使用深度学习和图嵌入进行恶意软件检测的方法,将Windows可移植可执行(PE)文件的API序列转换为调用图,调用图节点是API名称,有向边表示节点之间的调用关系。AMER^[19]等人提出一种基于API调用序列上下文关系的恶意软件检测方法,根据API序列中的上下文相似性对相关API进行分组,并生成API调用图。图节点是具有其组标签的API调用,并且每个边连接两个连续的API,实现了99%的平均检测精度。XIAO^[20]等人为了将API参数特征添加到图中,使用API调用名称和操作系统资源(即文件路径参数等)作为节点,边表示节点之间的引用关系。然而,由于API参数的复杂性,这些方法生成的图包含大量的节点和边,增加了分析的成本。LI^[21]等人提出一种用于准确检测和分类恶意软件的模型DMalNet,首先从API名称和参数中提取语义特征,然后从API调用序列中导出API调用图,将API调用之间的关系转换为图的结构信息,最后设计了一个图神经网络来实现恶意软件检测和分类。

以上3种检测方法关注API序列不同层面的特征,因此各有优势和不足。基于统计特征的检测方法,时间开销小、检测效率高,但攻击者可以通过添加冗余的API调用影响频率统计特征,从而绕过检测;基于语义特征的检测方法,能够捕捉API序列的细节特征,但通过插入额外的API序列,会误导模型产生错误分类结果;基于结构特征的检测方法,能够从整体上把握API序列行为特征,但攻击者可以通过修改API序

列来影响关键节点的特征,进而影响图神经网络的分类结果。为融合以上方法的优势,最大程度地提高恶意软件检测性能,本文提出了一种基于集成学习的恶意代码动态检测方法,根据API序列的不同特征,分别构建统计特征分析模块、语义特征分析模块和结构特征分析模块,最后融合各模块分析结果,得出最终检测结论。

2 基于集成学习的恶意代码动态检测方法

基于集成学习的恶意代码动态检测方法整体框架如图1所示,首先在受控的沙箱环境中执行恶意或良性软件原始exe文件,将活动记录存储为JSON报告。随后通过正则表达式匹配方式提取出所有API调用记录,经过3个模块进行分析,分别为统计特征分析模块、语义特征分析模块和结构特征分析模块。3个模块根据API序列的不同特征有针对性地进行检测,得到输出样本是否为恶意软件的概率,随后对3个概率取平均值,得出最终检测结果,若平均值大于0.5,则认为是恶意软件;否则,为良性软件。

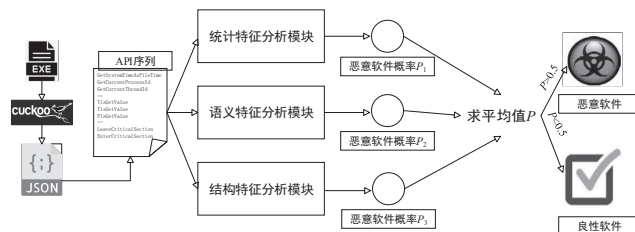


图1 基于集成学习的恶意代码动态检测方法整体框架

2.1 统计特征分析模块

统计特征分析模块具体结构如图2所示,首先采用n-gram算法,生成原始API序列的n-gram特征;随后计算每个特征的DF值,并从大到小进行排序,保留前1000个特征;最后通过随机森林分类器进行恶意代码检测。

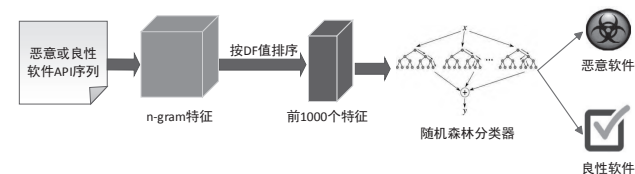


图2 统计特征分析模块结构

2.1.1 特征提取

n-gram算法是一种基于统计语言模型的算法,该模型基于马尔可夫假设:第 n 个词的出现只与前面 $n-1$ 个词相关,而与其他词不相关。基本思想是将文本里面的内容按照字节进行大小为 n 的滑动窗口操作,形成了长度为 n 的字节片段序列。每个字节片段称为gram,对所有gram的出现频度进行统计,并且按照事先设定好的阈值进行过滤,形成关键gram列表,也就是这个文本的向量特征空间,列表中的每种gram就是一个特征向量维度。API序列的n-gram特征提取方法如算法1所示。

算法1 API序列 n-gram特征提取算法

输入: 恶意或良性软件API序列 S

输出: n-gram特征 f_{ngram}

设置参数: 窗口大小 $n=2$, 阈值 $t=500$

初始化n-gram特征列表: $f_{ngram} \leftarrow NULL$

统计 S 中所有n-gram组合 $\{f_1, f_2, \dots, f_m\}$

计算每个组合出现次数 $\{freq_1, freq_2, \dots, freq_m\}$

for $i \leftarrow 1$ to m do

if $freq > t$: $f_{ngram}.append(f_i, freq_i)$

return f_{ngram}

由于不同特征的取值范围可能差别很大,容易因某个特征的取值范围过大或过小,从而出现极端值的情况,影响模型的训练效果。数据归一化可以将特征的方差减少到一定的范围内,消除特征间单位和尺度差异的影响,同时可以保留原始数据的分布信息。本文采取Min-Max归一化方法对数据进行归一化处理,使特征取值映射到 $[0,1]$ 的区间内,归一化公式如公式(1)所示,其中, $X_{\min}$ 为数据最小值, $X_{\max}$ 为数据的最大值。

$$X_{norm} = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad (1)$$

尽管n-gram算法已通过设定阈值的方式对特征数量进行了缩减,但特征维度仍达几千甚至上万,其中大量特征对于分类贡献很小,甚至会降低分类准确度,因此在采用机器学习算法进行分类前,合理选择特征

是十分重要的。文档频数(Document Frequency, DF)是较简单的一种特征选择算法,指的是在整个数据集中有多少个文本包含这个单词,DF的优点在于计算量很小,但在实际运用中却有很好的效果。对于恶意代码的一个n-gram特征而言,统计其文档频数即统计包含该特征的恶意代码文件数量。为快速高效地选择特征,依次统计每个n-gram特征的DF值,只保留DF值较大的前1000个特征。

2.1.2 模型设计

随机森林是一种重要的集成学习方法,广泛应用于分类和回归任务^[22]。随机森林的核心思想是组合多个决策树的预测结果,并通过平均值对其进行汇总,这种方法显著提高了模型的准确性和稳定性^[23]。本文采用2.1.1节提取的特征,在Python环境下使用sklearn库中的随机森林算法进行恶意代码检测,主要参数如表1所示。

表1 随机森林算法参数设置

参数	设置
n_estimators	100
criterion	gini
min_samples_split	2
min_samples_leaf	1
max_features	sqrt
random_state	42

2.2 语义特征分析模块

语义特征分析模块具体结构如图3所示,在预处理环节,对原始API序列进行去重,并将序列长度固定为500,随后输出到基于Transformer编码器的分类模型,进行词嵌入、位置编码和多头注意力机制等一系列操作,最后由全连接层输出恶意软件检测结果。

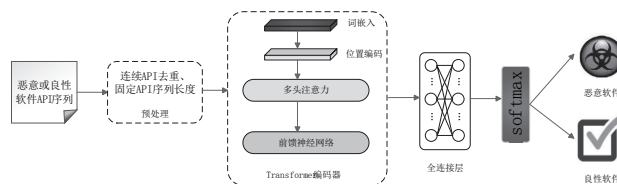


图3 语义特征分析模块结构

2.2.1 数据预处理

文献[15]的实验结果表明,在预处理环节对API

进行去重,不仅可以大大减小API序列长度,而且有助于提高恶意代码分类的准确率。因此本文参考文献[15]方法对API序列进行预处理,具体过程如图4所示。第一步,对于序列中连续重复多次的任何API调用,仅保留一次,生成一个新序列。第二步,从新序列中删除重复的二元子序列。随后依次删除重复的三元子序列和四元子序列。

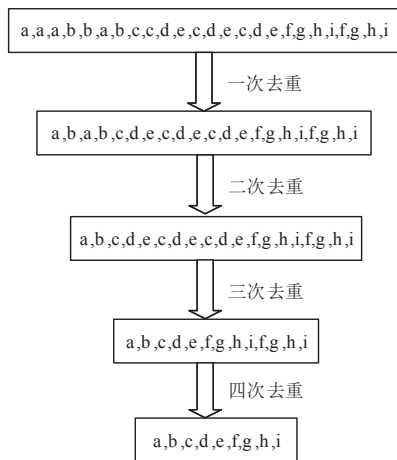


图4 API序列预处理过程

经过以上去重步骤后,固定序列长度为500个API,若API序列长度大于500,则删除超出范围的API记录;若长度不足500,则用标记符“<pad>”进行填充。

2.2.2 模型设计

Transformer模型由VASWANI^[24]等人于2017年提出,旨在解决传统的序列到序列模型在处理长距离依赖问题上的不足。Transformer通常包括一个编码器和一个解码器,编码器用于处理输入序列,而解码器负责生成输出序列,适用于各种序列到序列的任务。由于本文研究的是API序列的二分类问题,只需用到Transformer的编码器部分,编码器由词嵌入、位置编码、注意力机制和前馈神经网络组成。

在词嵌入环节,本文采用了Word2Vec^[25]词嵌入方法,Word2Vec在来自训练集的API调用序列的大型语料库上进行训练,并产生固定大小的向量空间。在训练之后,输入语料库中的每个不同的API名称被表示为语义特征向量。在位置编码环节,采用VASWANI^[24]

等人提出的方法,为API序列中的每个位置创建一组具有不同频率的正弦和余弦函数,如公式(2)和公式(3)所示。

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (2)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (3)$$

其中, $PE_{(pos,i)}$ 是序列中位置 pos 处标记的位置编码的第 i 维, d 是模型的维数。 $PE_{(pos,2i)}$ 和 $PE_{(pos,2i+1)}$ 项分别用于偶数和奇数维度 i 。然后将这些值添加到嵌入向量 e_{pos} 中,以将位置信息合并到序列。

自注意力机制(Self-Attention)是Transformer模型中的关键组成部分,其核心思想是,对于输入序列中的每个位置,模型可以根据该位置与其他位置的关系,自动计算出一个权重向量,用于表示该位置在整个序列中的重要性。这种权重向量可以看作对输入序列中各个位置的注意力分布,即模型决定关注序列中哪些位置的信息。自注意力操作如公式(4)所示,其中, Q 、 K 和 V 分别为查询、键和值,用作自注意力层的输入, d_k 是键的维度。

$$Attention(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (4)$$

为了进一步提高模型的表达能力,Transformer编码器引入了多头注意力机制。通过在不同的子空间上进行注意力计算,多头注意力机制使得模型能够关注输入序列中不同层次和不同方面的信息,从而更好地进行特征提取和表示学习。多头注意力机制的计算过程如图5所示(以8个头为例),如公式(5)所示。

$$\begin{aligned} MultiHead(Q,K,V) = & \\ Concat\left(softmax\left(\frac{Q_0K_0^T}{\sqrt{d_k}}\right)V_0, softmax\left(\frac{Q_1K_1^T}{\sqrt{d_k}}\right)V_1, \dots, \right. & \\ \left. softmax\left(\frac{Q_hK_h^T}{\sqrt{d_k}}\right)V_h\right)W^O & \end{aligned} \quad (5)$$

除了自注意力机制外,Transformer编码器还包含一种全连接的前馈神经网络结构。前馈神经网络首先对输入序列进行线性变换,得到一组新的表示向量,

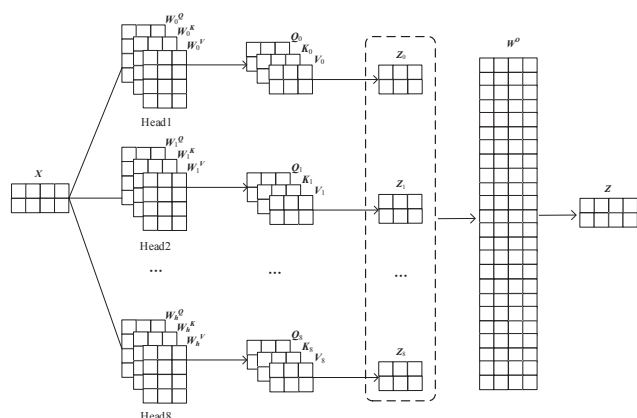


图5 多头注意力机制计算过程

这组表示向量随后被用于计算注意力权重。前馈网络与自注意力机制相结合,使得模型不仅能够关注输入序列中的不同位置,还能够通过非线性变换捕捉更复杂的特征。这种结合使得Transformer模型在处理序列数据时具有很高的效率和有效性。

API序列在经过Transformer编码器处理后,不能直接得到分类结果,因此需要在编码器后添加全连接层和softmax层,全连接层每个神经元都与上一层的所有神经元相连,将前面各层提取到的特征进行权重整合,随后softmax层接收全连接层的输出作为输入,并将其转化为每个类别的概率值,这样模型就可以输出恶意软件和良性软件的预测概率。

2.3 结构特征分析模块

结构特征分析模块具体设计如图6所示,首先根据恶意或良性软件API序列构建一个有向图,得到节点和边的信息,随后输入图神经网络进行处理,依次经过GINConv层、池化层和全连接层,得到恶意代码检测结果。

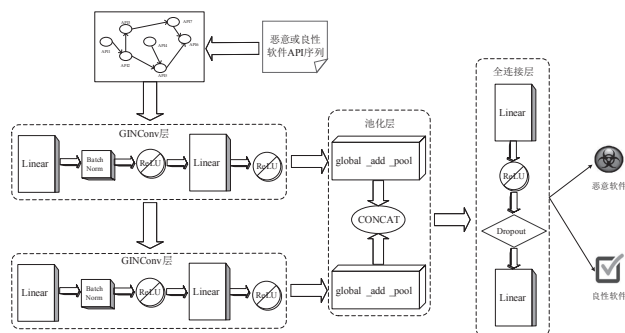


图6 结构特征分析模块

2.3.1 数据预处理

统计训练集上出现的所有API名称,从1开始逐个进行编号,对于每个API序列,用编号替换原API名称,每个编号代表一个节点,重复出现的编号按一个节点处理,前后相邻的两个编号构成一条有向边,重复出现的边按照一条边处理。经过以上步骤,便可得到一个API序列的有向图信息,根据API序列生成有向图的具体过程如算法2所示。

算法2 API序列转化为有向图

输入: 恶意或良性软件API序列 S

输出: 有向图节点信息 G_n , 边信息 G_e

计算API序列长度 $l = \text{len}(S)$

初始化节点和边信息列表 $G_n \leftarrow \text{NULL}$, $G_e \leftarrow \text{NULL}$

for $i \leftarrow 1$ to l do

$S'_i \leftarrow \text{getnumber}(S_i)$

if S'_i not in G_n : $G_n.append(S'_i)$

for $j \leftarrow 1$ to $l-1$ do

$\text{newedge} \leftarrow (G_n(j), G_n(j+1))$

if newedge not in G_e : $G_e.append(\text{newedge})$

return G_n, G_e

2.3.2 模型设计

XU^[26]等人于2018年提出了一种图神经网络模型,即GIN,GIN相较于其他GNN模型具备高表征能力,几乎能够完美拟合训练数据。文献[26]中证明了当节点特征 X 可数时,将节点特征的聚合方式 (Aggregate) 设置为sum,邻域特征与中心节点特征的融合系数设置为 $1+\varepsilon$,会存在一个函数 $f(x)$ 使得聚合函数为单射函数,如公式(6)所示。

$$h(c, X) = (1 + \varepsilon) \cdot f(c) + \sum_{x \in X} f(x) \quad (6)$$

其中, $h(c, X)$ 表示节点 c 的特征和它的邻居节点 X 的特征的聚合结果, ε 是一个可调参数,表示中心节点特征的权重, $f(c)$ 表示节点 c 的特征向量, $\sum_{x \in X} f(x)$ 表示节点 c 的所有邻居节点特征的和。

同时,文献[26]进一步证明了对于任意聚合函数 $g(c, X)$,在满足单射性的条件下可以分解为

$$g(c, X) = \varphi(1 + \varepsilon) \cdot f(c) + \sum_{x \in X} f(x) \quad (7)$$

其中, φ 是一个函数, 可以是任何非线性激活函数, 如 ReLU, 通过引入这种非线性变换, 增强了模型的表达能力, 能够捕捉到更加复杂的图结构和节点之间的关系。

然后借助多层感知机强大的拟合能力来学习公式(7)中的 φ 和 f , 最后得到基于 MLP+SUM 的 GIN 模型。公式(8)是 GIN 中的节点更新公式, 描述了如何通过多层感知机和邻居节点信息来更新每层的节点特征。

$$\mathbf{h}_v^{(k)} = \text{MLP}^{(k)}((1 + \varepsilon^{(k)}) \cdot \mathbf{h}_v^{(k-1)} + \sum_{u \in N(v)} \mathbf{h}_u^{(k-1)}) \quad (8)$$

其中, $\mathbf{h}_v^{(k)}$ 表示第 k 层的节点 v 的特征向量, $\text{MLP}^{(k)}$ 表示第 k 层的多层感知机, $\varepsilon^{(k)}$ 表示第 k 层的参数, $\mathbf{h}_v^{(k-1)}$ 表示第 $k-1$ 层的节点 v 的特征向量, $\sum_{u \in N(v)} \mathbf{h}_u^{(k-1)}$ 表示节点 v 在第 $k-1$ 层的所有邻居节点的特征向量的和, $N(v)$ 表示节点 v 的邻居节点集合。

对于每轮迭代产生的节点特征求和, 然后拼接作为图的特征表示, 如公式(9)所示。

$$\mathbf{h}_G = \text{CONCAT}(\sum_{v \in G} \mathbf{h}_v^{(k)} | k = 0, 1, \dots, K) \quad (9)$$

其中, CONCAT 表示连接操作, 将各层的特征向量连接起来, $\sum_{v \in G} \mathbf{h}_v^{(k)}$ 表示图 G 中所有节点在第 k 层的特征向量的和, 称为全局求和池化, 这种方式能够综合考虑各层次的信息, 使得最终的图特征表示更加丰富和全面。

本文采用 PyTorch Geometric 库的 GINConv 层来实现以上 GIN 的设计, 在 GINConv 层中自定义一个 MLP, 用于学习节点的特征表示, 包括全连接层、批量归一化层和激活函数层。与文献[26]中采用 5 个串联的 MLP 结构不同, 本文仅使用两个 MLP 进行串联, 从而简化了模型结构, 既保持了模型的非线性拟合能力, 又能有效减少计算资源的消耗和过拟合的风险。

3 实验与分析

3.1 数据集与实验环境

本文采用了 TRIZNA^[27] 于 2022 年发布的 Speakeasy 数据集, 该数据集使用 Windows 内核模拟器 Speakeasy v1.5.9^[28] 生成, 包含来自总共约 93500 个样本的行为报

告, 合法和恶意报告均采用 JSON 格式, 恶意样本属于 7 种不同的恶意软件类型。图 7 为一个 JSON 样本文件的部分内容, 由于实验中采用的是 API 序列进行分类, 故需要通过正则表达式提取所有 “api_name” 里包含的 API 名称。

```
"apis": [
  {
    "pc": "0x409a49",
    "api_name": "MSVCRT.__set_app_type",
    "args": [
      "0x2"
    ],
    "ret_val": null
  },
  {
    "pc": "0x409a5e",
    "api_name": "MSVCRT.__p_fmode",
    "args": [],
    "ret_val": "0x4610"
  },
  {
    "pc": "0x409a6c",
    "api_name": "MSVCRT.__p_commode",
    "args": [],
    "ret_val": "0x4620"
  },
  ...
]
```

图 7 JSON 样本文件内容

Speakeasy 数据集的训练集数据在 2022 年 1 月收集, 测试集数据在 2022 年 4 月收集, 这种时间分离有助于检查恶意软件行为中的概念漂移。数据集结构如表 2 所示, 实验环境如表 3 所示。

表 2 Speakeasy 数据集样本数量分布

类型	训练集 / 个	测试集 / 个
clean	24434	7944
backdoor	11062	1940
coinminer	6891	1684
dropper	8243	252
keylogger	4378	1041
ransomware	9627	2139
rat	1697	1258
trojan	8733	1085

表 3 实验环境配置

实验环境	具体配置
操作系统	Windows 11
CPU	Intel(R) Core(TM) i7-13620H CPU @ 2.40GHz
内存	16GB
硬盘	1TB
显卡	NVIDIA GeForce RTX 4050
开发框架	PyTorch
开发语言	Python 3.10

为提高模型的鲁棒性, 从训练集的恶意软件和良性软件中各选取 50% 的样本进行数据增强, 方法是每个样本随机添加 1~100 之间数量不等的 API, 得到新

的样本，于是训练集规模扩大1.5倍。

3.2 评价指标

本文实验选用了准确率 *Accuracy*、*F1* 值、*TPR* 和 *AUC* 这4个指标进行评价，各指标的定义分别如公式 (10)~公式 (13) 所示。

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (10)$$

$$F1 = 2 \times \frac{(TP / (TP + FP)) \times (TP / (TP + FN))}{TP / (TP + FP) + TP / (TP + FN)} \quad (11)$$

$$TPR = \frac{TP}{TP + FN} \quad (12)$$

$$AUC = \int_0^1 TPR d(FPR) \quad (13)$$

其中，*TP* 表示实际的阳性恶意代码样本被正确预测为阳性样本，*FP* 表示实际的阴性恶意代码样本被错误地预测为阳性样本。类似的，*TN* 表示实际的阴性恶意代码样本被正确地预测为阴性样本，*FN* 表示实际的阳性恶意代码样本被错误地预测为阴性样本。*Accuracy* 表示所有预测正确的样本（包含正例或负例均预测正确）占总样本的比例。*F1* 值是精确率和召回率的调和平均，精确率表示预测为正例的样本中，真正的正例占预测为正例样本的比例。召回率表示实际为正例的样本中，被预测正确的正例占总实际正例样本的比例。*F1* 值越大说明模型质量越高。*TPR* 表示真阳性率，通常和假阳性率 *FPR* 配合使用，本文采用 *FPR* 为 10^{-3} 时的 *TPR* 作为评价指标，因为在实际应用中进行恶意软件检测往往要求较低的误报率。*AUC* 表示 ROC 曲线下的面积大小，其越大表明模型的性能越好。

3.3 实验结果分析

3.3.1 各模块性能分析

统计特征分析模块采用 *n*-gram 特征作为恶意代码检测依据，为确定 *n* 的最佳取值，依次提取 API 序列的 2-gram、3-gram、4-gram 和 5-gram 特征进行实验，受实验环境计算能力限制，不再提取 6-gram 以上的特征。实验结果如表4所示。

由表4可以看出，无论是在训练集还是测试集上，

表4 不同 *n* 值 *n*-gram 特征效果对比

	训练集				测试集			
	<i>Accuracy</i>	<i>F1</i> - 值	<i>TPR</i> (<i>FPR</i> = 10^{-3})	<i>AUC</i>	<i>Accuracy</i>	<i>F1</i> - 值	<i>TPR</i> (<i>FPR</i> = 10^{-3})	<i>AUC</i>
2-gram	0.9592	0.9701	0.7962	0.9918	0.8432	0.8446	0.3879	0.9360
3-gram	0.9515	0.9646	0.7616	0.9879	0.8334	0.8363	0.3806	0.9351
4-gram	0.9348	0.9529	0.7207	0.9793	0.8317	0.8367	0.3856	0.9284
5-gram	0.9159	0.9404	0.6827	0.9665	0.8261	0.8386	0.3694	0.9202

采用 2-gram 特征时整体效果最佳，因此后续都以 2-gram 特征作为研究对象。此外还可以发现，测试集上各项指标明显低于训练集，为找出导致这种差距的原因，对 Speakeasy 数据集使用的 API 进行了统计分析，发现训练集出现的 API 名称共 2010 个，而整个数据集出现的 API 名称共 2122 个，这意味着测试集中有 112 个 API 未在训练集出现，这说明测试集的数据分布规律已发生变化，属于概念漂移现象，这是影响模型在测试集上性能的主要原因。

语义特征分析模块采用了基于 Transformer 编码器的模型进行 API 序列分类，模型训练过程中的准确率和 *F1* 值变化曲线如图8和图9所示。

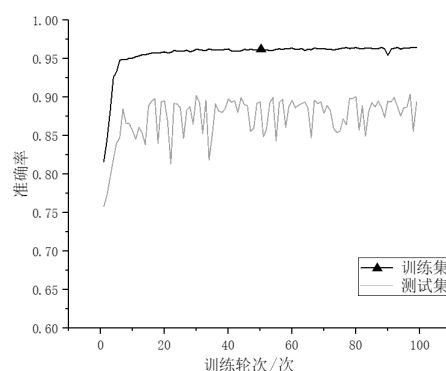


图8 语义特征分析模块的准确率变化曲线

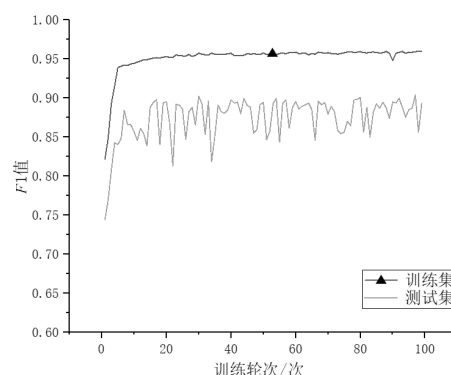


图9 语义特征分析模块的 *F1* 值变化曲线

由图8和图9可以观察出,在训练集上,模型收敛速度较快,在前10个训练轮次里准确率和F1值都上升到95%以上,且曲线比较平稳。在测试集上,随着轮次的增加,准确率和F1值整体上也呈上升趋势,但和训练集相比性能有一定差距,平均差距为8%,且曲线波动也比较明显,说明也受到了概念漂移现象的影响。

结构特征分析模块采用了GIN模型进行API序列分类,模型训练过程中的准确率和F1值变化曲线如图10和图11所示。

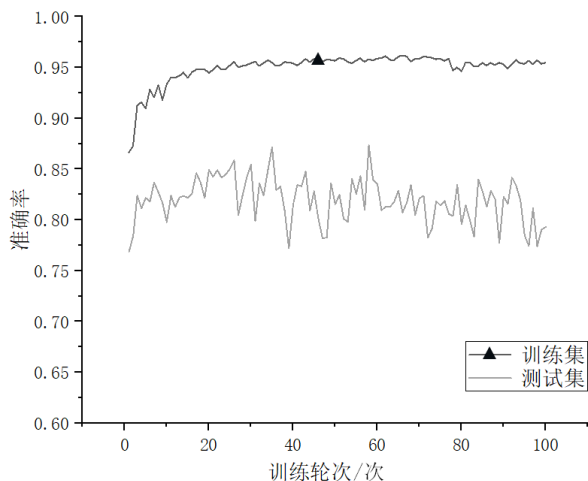


图 10 结构特征分析模块准确率变化曲线

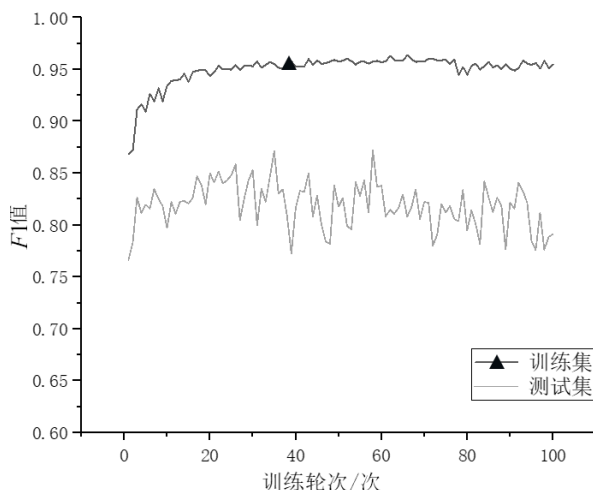


图 11 结构特征分析模块 F1 值变化曲线

由图10和图11可以观察出,模型在训练集上准确率和F1值最终都稳定在95%左右,在测试集上曲线波动较为明显,与训练集相比性能平均差距为13%,说

明受概念漂移现象的影响更大。经过分析有两方面原因,一方面,测试集出现了训练集中未曾出现的API,模型无法将其转换为节点特征;另一方面,测试集中有的样本使用的API数量较少,甚至有时只使用了一个API,因此将其转化为图结构时,只有一个节点信息,没有任何边的信息,这种情况不利于图神经网络进行分类。

3.3.2 集成学习性能分析

3.3.1节的实验结果表明,当单个模块独立运行时,在测试集上的性能均不够理想。考虑到不同模块关注不同层面的特征,若将不同模块的优势互补,便有可能增强模型性能。对于统计特征分析模块,采用训练并保存好的随机森林分类器,在测试集上进行预测,不输出预测的具体类别,只输出预测为恶意软件的概率。对于语义特征分析模块,由于其在测试集上的性能存在波动,仅取一个训练好的模型可能不足以代表该模块的效果,因此取10个训练好的模型依次在测试集上进行预测,得到是否为恶意软件的概率,随后对10个概率取平均值,作为该模块的输出。结构特征分析模块在测试集上的性能也存在波动,操作方法同语义特征分析模块。最后对3个模块输出的概率取平均值,作为恶意软件检测结果,若大于0.5,则检测为恶意软件;否则,为良性软件。

为验证以上集成学习方法的有效性,首先使各模块独立运行,随后两两进行组合,最后集成3个模块共同运行,依次在测试集上检验效果。实验结果如表5所示,模块A代表统计特征分析模块,模块B代表语义特征分析模块,模块C代表结构特征分析模块。

表 5 不同集成方式性能对比

集成方式	评价指标			
	Accuracy	F1 值	TPR (FPR=10 ⁻³)	AUC
仅模块 A	0.8432	0.8446	0.3879	0.9360
仅模块 B	0.8711	0.8718	0.4112	0.9518
仅模块 C	0.8300	0.8390	0.3467	0.8937
模块 A+B	0.9255	0.9281	0.5242	0.9745
模块 A+C	0.8827	0.8862	0.4448	0.9573
模块 B+C	0.9121	0.9163	0.4926	0.9648
模块 A+B+C	0.9338	0.9352	0.6370	0.9821

由表5可以看出,采用单模块进行检测,准确率和F1值均在90%以下,TPR和AUC的值也较低,采用集成

学习后,各项指标均有提升,充分说明通过集成可以使各模块优势互补,从而得到更好的检测效果。集成3个模块运行时性能提升最大,随后是集成模块A和B,再次是集成模块B和C,最后是集成模块A和C,可见模块B对集成学习的性能贡献最大,模块A次之,模块C贡献最小,这与单个模块独立运行时的性能恰好相对应。

为了更清楚地观察模型分类结果的细节,绘制了混淆矩阵如图12~图18所示。

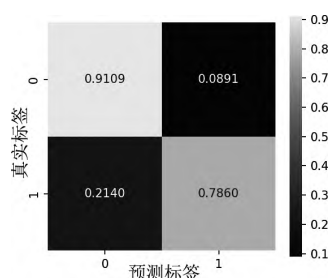


图12 模块A对应混淆矩阵

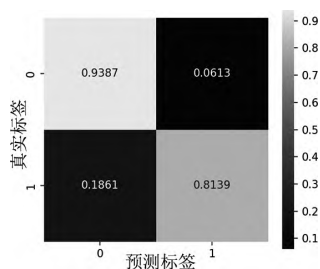


图13 模块B对应混淆矩阵

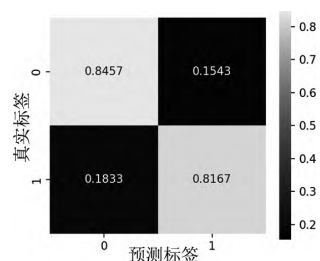


图14 模块C对应混淆矩阵

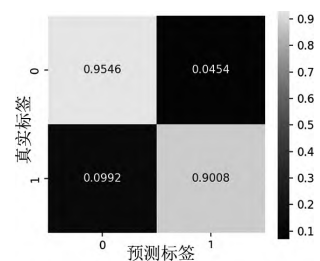


图15 模块A+B对应混淆矩阵

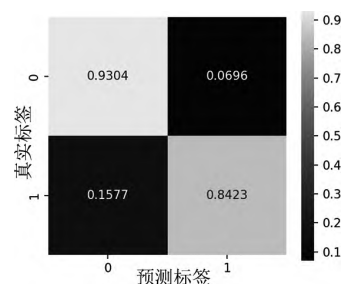


图16 模块A+C对应混淆矩阵

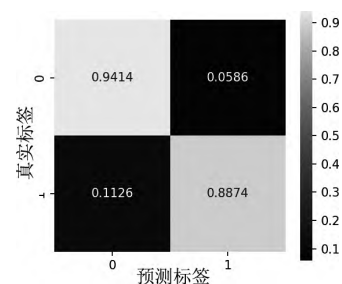


图17 模块B+C对应混淆矩阵

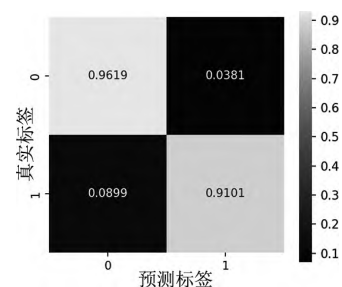


图18 模块A+B+C对应混淆矩阵

由图12~图18可以看出,集成3个模块后,在分类细节方面表现最好,TP和TN的值最大,FP和FN的值最小。

3.3.3 与现有研究方法对比

为进一步验证本文方法的有效性,设置对比实验,与近年来其他采用Speakeasy数据集进行恶意代码检测的方法进行比较,结果如表6所示。

表6 本文方法和其他方法效果对比

方法	评价指标			
	Accuracy	F1 值	TPR (FPR=10 ⁻³)	AUC
文献[12]方法	0.7014	0.6465	0.2152	0.8879
文献[29]方法	0.8786	0.8792	0.4250	0.9528
文献[17]方法(BPE)	0.9104	0.9136	0.5213	0.9657
文献[17]方法(Whitespace)	0.9053	0.9058	0.5703	0.9664
本文方法	0.9338	0.9352	0.6370	0.9821

文献[12]提出的深度神经网络架构采用多个Gated-CNN来变换每个API调用的提取特征,通过双向LSTM对输出进行进一步处理,以学习API调用之间的顺序相关性。文献[29]借鉴文档分类领域的思想,采用CNN、Bi-LSTM和Attention搭建模型,使用报告中存在的单词序列来预测报告是否来自恶意二进制文件。文献[17]除了从JSON文件中提取API名称和参数外,还提取网络活动和文件操作等信息作为恶意软件检测的依据,搭建了基于Transformer自注意力机制的模型Nebula,并采用BPE和Whitespace两种分词算法分别进行实验。以上方法有个共同点,都借鉴了自然语言处理的方法,将API调用视为一个单词序列,从序列中通过挖掘语义特征进行恶意软件检测,而本文提出的集成学习方法不仅考虑了语义特征,也考虑了统计特征和结构特征,因此在各项评价指标上均有一定优势。

3.3.4 对抗性攻击实验

目前,针对API序列进行攻击的常用方法有随机扰动和良性扰动两种,这两种方法均由ROSENBERG^[30]等人提出,随机扰动在恶意软件API序列中随机选取多个位置,在这些位置随机插入新的API,良性扰动采用SeqGAN模型^[31]生成模仿良性软件的API序列,随后从中随机选取一个片段插入原始API序列中。这两种方法只考虑插入API,而不考虑删除或修改API,主要是为了避免破坏恶意软件原有的功能,此外新插入的API来源于杜鹃沙箱中监控的314个API^[32],这些API结合适当的参数,均可视为无操作API,这样也避免了对原API序列功能的影响。

为检验本文方法的鲁棒性,从Speakeasy数据集的测试集中选取1000个模型预测正确的恶意软件样本,分别采用随机扰动和良性扰动的方法,在不同扰动量下各生成1000个对抗样本进行测试,结果如图19所示。由图19可以观察出,当扰动量介于0~100之间时,两种方法的攻击成功率都接近于0,这与3.1节的数据增强方式有直接关系,随着扰动量的增加,攻击成功率呈上升趋势,随机扰动的攻击成功率最高为3%左右,

良性扰动的攻击成功率最高为18%左右,这说明模型对随机扰动的鲁棒性较好,对良性扰动也有一定抵抗能力。

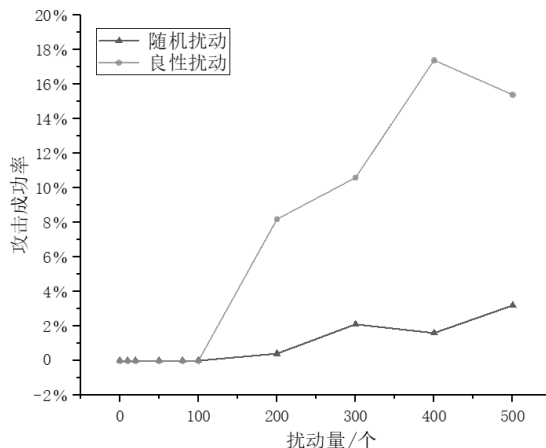


图 19 对抗性攻击实验结果

为了探究数据增强对模型鲁棒性的影响,设置了消融实验,在未经过数据增强的情况下重新训练各个模块,训练完毕后再次对1000个对抗样本进行测试,结果如图20所示。由图20可以观察出,即使在扰动量特别小的情况下,攻击效果也十分明显,随着扰动量的增加,随机扰动攻击成功率最高为63%左右,良性扰动攻击成功率最高为70%左右,可见对数据集进行数据增强,对于提高模型的鲁棒性意义十分重大。

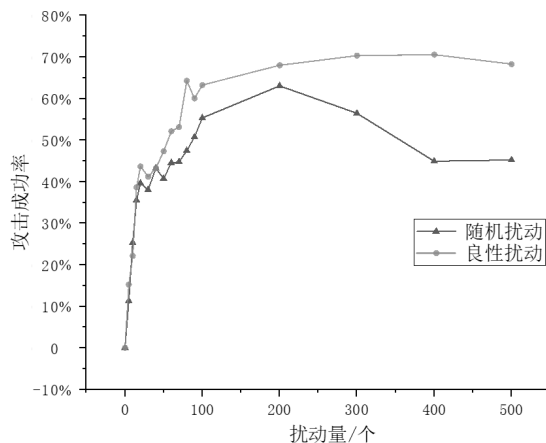


图 20 未进行数据增强时对抗性攻击实验结果

此外,为了深入探究单个模块对集成学习鲁棒性的影响,设置了消融实验进行效果对比,结果如图21和图22所示。模块A代表统计特征分析模块,模块B

代表语义特征分析模块,模块C代表结构特征分析模块。

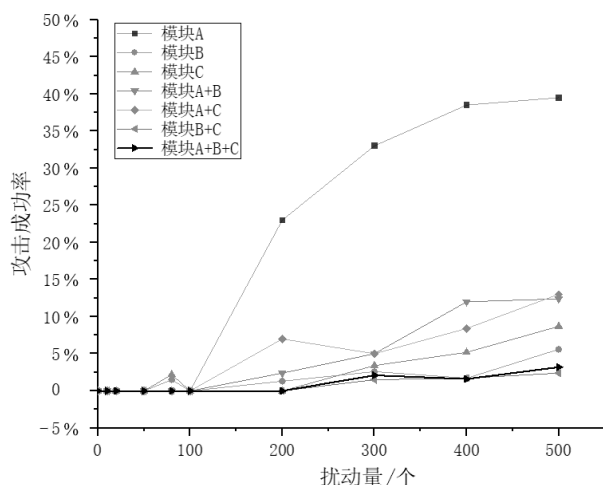


图 21 不同模块鲁棒性对比（随机扰动）

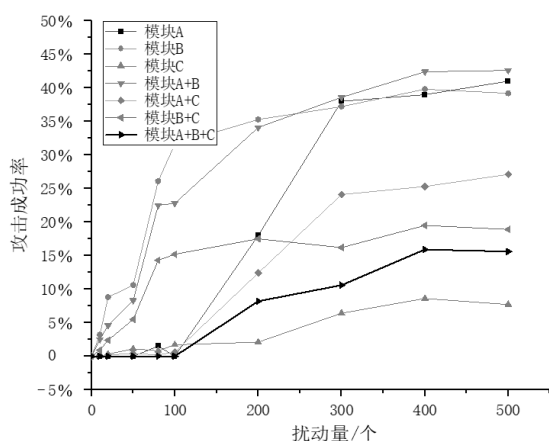


图 22 不同模块鲁棒性对比（良性扰动）

从图 21 可以观察到，当扰动量较小时，随机扰动对 3 个模块的影响都较小，攻击成功率趋于 0，随着扰动量的增加，模块 A 受到的影响最大，而模块 B 和模块 C 受到的影响较小。可见对于随机扰动，模块 A 鲁棒性较差，模块 B 和模块 C 鲁棒性较好。集成模块 B 和模块 C 运行时，效果最佳，集成 3 个模块的运行效果次之。

从图 22 可以观察到，当良性扰动量较小时，对模块 B 有一定影响，对模块 A 和模块 C 影响较小，随着扰动量的增加，模块 A 和模块 B 都受到了较大影响，但模块 C 受到的影响较小。可见对于良性扰动，模块 A 和模块 B 鲁棒性较差，模块 C 鲁棒性较好。模块 C 单

独运行时鲁棒性最佳，集成 3 个模块的运行效果次之。

无论是随机扰动还是良性扰动，集成 3 个模块运行时鲁棒性并不一定是最优，但都能取得比较满意的效果，且 3.3.2 节中的实验结果表明，集成 3 个模块运行时准确性最佳，说明这种集成方法在提高模型精度的同时，能保持较好的鲁棒性。

4 结束语

本文提出一种基于集成学习的恶意代码动态检测方法，根据 API 序列的不同特征，分别构建了 3 个模块针对性地进行恶意代码检测。统计特征分析模块提取 API 序列的 2-gram 特征，采用随机森林分类器进行恶意代码检测，语义特征分析模块采用基于 Transformer 编码器的深度学习模型，挖掘 API 序列的语义特征进行恶意代码检测，结构特征分析模块将每个 API 序列转化为有向图，采用 GIN 聚合节点和边的信息进行恶意代码检测。最后融合 3 个模型的输出概率，得出最终检测结果。这种集成学习方法能够结合不同模型的优势，在提高模型性能的同时，保持了较好的鲁棒性，能够有效应对常见的两种针对 API 序列的攻击方法。下一步将在更多数据集上开展实验，验证集成学习方法的泛化性能，并采用更先进的对抗样本生成方法测试模型的鲁棒性，从而在实际应用中发挥价值。

参考文献：

- [1] Kaspersky Security Network. The Mobile Malware Threat Landscape in 2023[EB/OL]. (2024-02-06)[2024-09-10]. <https://securelist.com/mobile-malware-report-2023/111964>.
- [2] LI Sicong, WANG Jian, SONG Yafei, et al. Malicious Code Classification Method Based on BiTCN-DLP[J]. Netinfo Security, 2023, 23(11): 104-117. 李思聪,王坚,宋亚飞,等.基于 BiTCN-DLP 的恶意代码分类方法[J]. 信息安全, 2023, 23(11): 104-117.
- [3] SUN Hongzhe, WANG Jian, WANG Peng, et al. Network Intrusion Detection Method Based on Attention-BiTCN[J]. Netinfo Security, 2024, 24(2): 309-318. 孙红哲,王坚,王鹏,等.基于 Attention-BiTCN 的网络入侵检测方法[J]. 信息安全, 2024, 24(2): 309-318.
- [4] ZHANG Dandan, SONG Yafei, LIU Shu. MalMKNet: A Multi-Scale Convolutional Neural Network Used for Malware Classification[J]. Acta Electronica Sinica, 2023, 51(5): 1359-1369. 张丹丹,宋亚飞,刘曙. MalMKNet: 一种用于恶意代码分类的多尺度卷积神经网络[J]. 电子学报, 2023, 51(5): 1359-1369.

- [5] GALLORO N, POLINO M, CARMINATI M, et al. A Systematical and Longitudinal Study of Evasive Behaviors in Windows Malware[EB/OL]. (2021-12-05)[2024-09-10]. <https://doi.org/10.1016/j.cose.2021.102550>.
- [6] CHAI Yuhua. Research on Key Technologies for Malware Classification in Open World[D]. Guangzhou: Guangzhou University, 2023.
- 柴瑜晗. 开放场景下恶意软件分类关键技术研究[D]. 广州: 广州大学, 2023.
- [7] QIAO Yong, YANG Yueyang, JI Lin, et al. Analyzing Malware by Abstracting the Frequent Itemsets in API Call Sequences[C]// IEEE. 2013 12th IEEE International Conference on Trust, Security and Privacy in Computing and Communications. New York: IEEE, 2013: 265-270.
- [8] UPPAL D, SINHA R, MEHRA V, et al. Malware Detection and Classification Based on Extraction of API Sequences[C]// IEEE. 2014 International Conference on Advances in Computing, Communications and Informatics (ICACCI). New York: IEEE, 2014: 2337-2342.
- [9] ALAZAB M, ALAZAB M, SHALAGINOV A, et al. Intelligent Mobile Malware Detection Using Permission Requests and API Calls[J]. Future Generation Computer Systems, 2020, 107: 509-521.
- [10] KOLOSINAJI B, ZARRAS A, WEBSTER G, et al. Deep Learning for Classification of Malware System Call Sequences[C]//Springer. Advances in Artificial Intelligence: 29th Australasian Joint Conference (AI 2016). Heidelberg: Springer, 2016: 137-149.
- [11] AGRAWAL R, STOKES J W, MARINESCU M, et al. Neural Sequential Malware Detection with Parameters[C]// IEEE. 2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). New York: IEEE, 2018: 2656-2660.
- [12] ZHANG Zhaoqi, QI Panpan, WANG Wei. Dynamic Malware Analysis with Feature Engineering and Feature Learning[C]// AAAI. Proceedings of the AAAI Conference on Artificial Intelligence. New York: AAAI, 2020, 34(1): 1210-1217.
- [13] LI Ce, LYU Qiuqian, LI Ning, et al. A Novel Deep Framework for Dynamic Malware Detection Based on API Sequence Intrinsic Features[EB/OL]. (2022-03-17)[2024-09-10]. <https://doi.org/10.1016/j.cose.2022.102686>.
- [14] ZHANG Sanfeng, WU Jiahao, ZHANG Mengzhe, et al. Dynamic Malware Analysis Based on API Sequence Semantic Fusion[EB/OL]. (2023-05-26)[2024-09-10]. <https://doi.org/10.3390/app13116526>.
- [15] DEMIRKIRAN F, ÇAYIR A, UNAL U, et al. An Ensemble of Pre-Trained Transformer Models for Imbalanced Multiclass Malware Classification[EB/OL]. (2022-08-06)[2024-09-10]. <https://doi.org/10.1016/j.cose.2022.102846>.
- [16] LI Yaping, LI Yuancheng. IoT Malware Threat Hunting Method Based on Improved Transformer[J]. International Journal of Network Security, 2023, 25(2): 267-276.
- [17] TRIZNA D, DEMETRIO L, BIGGIO B, et al. Nebula: Self-Attention for Dynamic Malware Analysis[J]. IEEE Transactions on Information Forensics and Security, 2024, 19: 6155-6167.
- [18] JIANG Haodi, TURKI T, WANG J T L. DLGraph: Malware Detection Using Deep Learning and Graph Embedding[C]//IEEE. 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA). New York: IEEE, 2018: 1029-1033.
- [19] AMER E, ZELINKA I. A Dynamic Windows Malware Detection and Prediction Method Based on Contextual Understanding of API Call Sequence[EB/OL]. (2020-02-20)[2024-09-10]. <https://doi.org/10.1016/j.cose.2020.101760>.
- [20] XIAO Fei, LIN Zhaowen, SUN Yi, et al. Malware Detection Based on Deep Learning of Behavior Graphs[EB/OL]. (2019-02-11)[2024-09-10]. <https://doi.org/10.1155/2019/8195395>.
- [21] LI Ce, CHENG Zijun, ZHU He, et al. DMALNet: Dynamic Malware Analysis Based on API Feature Engineering and Graph Learning[EB/OL]. (2022-08-21)[2024-09-10]. <https://doi.org/10.1016/j.cose.2022.102872>.
- [22] DONG Shishi, HUANG Zhexue. A Brief Theoretical Overview of Random Forests[J]. Journal of Integration Technology, 2013, 2(1): 1-7.
- [23] PARMAR A, KATARIYA R, PATEL V. A Review on Random Forest: An Ensemble Classifier[C]// Springer. International Conference on Intelligent Data Communication Technologies and Internet of Things (ICICI). Heidelberg: Springer, 2019: 758-763.
- [24] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is All You Need[J]. Advances in Neural Information Processing Systems, 2017, 30: 5998-6008.
- [25] MIKOLOV T, CHEN Kai, CORRADO G, et al. Efficient Estimation of Word Representations in Vector Space[EB/OL]. (2013-09-07)[2024-09-10]. <https://arxiv.org/pdf/1301.3781>.
- [26] XU Keyulu, HU Weihua, LESKOVEC J, et al. How Powerful are Graph Neural Networks?[EB/OL]. (2018-10-04)[2024-09-10]. <https://arxiv.org/pdf/1810.00826.pdf>.
- [27] TRIZNA D. Quo Vadis: Hybrid Machine Learning Meta-Model Based on Contextual and Behavioral malware Representations[C]//ACM. Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security. New York: ACM, 2022: 127-136.
- [28] MANDIANT. Speakeasy: Portable, Modular, Binary Emulator Designed to Emulate Windows Kernel and User Mode Malware[EB/OL]. (2021-10-11)[2024-09-10]. <https://github.com/mandiant/speakeasy>.
- [29] JINDAL C, SALLS C, AGHAKHANI H, et al. Neurlux: Dynamic Malware Analysis without Feature Engineering[C]//ACM. Proceedings of the 35th Annual Computer Security Applications Conference. New York: ACM, 2019: 444-455.
- [30] ROSENBERG I, SHABTAI A, ELOVICI Y, et al. Query-Efficient Black-Box Attack against Sequence-Based Malware Classifiers[C]//ACM. Proceedings of the 36th Annual Computer Security Applications Conference. New York: ACM, 2020: 611-626.
- [31] YU Lantao, ZHANG Weinan, WANG Jun, et al. Seqgan: Sequence Generative Adversarial Nets with Policy Gradient[EB/OL]. (2017-02-13)[2024-09-10]. <https://doi.org/10.1609/aaai.v31i1.10804>.
- [32] Cuckoo Sandbox. Cuckoo Sandbox Hooked APIs and Categories[EB/OL]. (2019-08-24)[2024-09-10]. <https://github.com/cuckoosandbox/cuckoo/wiki/Hooked-APIs-and-Categories>.